

Лабораторная работа №2
Компьютерный практикум по статистическому анализу данных

Исаев Б. А.

2025

Российский университет дружбы народов имени Патриса Лумумбы, Москва, Россия

Кортежи

```
[1]: # пустой кортеж:
      ()

[1]: ()

[2]: # кортеж из элементов типа String:
      favoritelang = ("Python", "Julia", "R")

[2]: ("Python", "Julia", "R")

[3]: # кортеж из целых чисел:
      x1 = (1, 2, 3)

[3]: (1, 2, 3)

[4]: # кортеж из элементов разных типов:
      x2 = (1, 2.0, "tmp")

[4]: (1, 2.0, "tmp")

[5]: # именованный кортеж:
      x3 = (a=2, b=1+2)

[5]: (a = 2, b = 3)
```

Рис. 1: Примеры кортежей

Кортежи

```
[7]: # длина кортежа x2:  
length(x2)
```

```
[7]: 3
```

```
[8]: # обратиться к элементам кортежа x2:  
x2[1], x2[2], x2[3]
```

```
[8]: (1, 2.0, "tmp")
```

```
[9]: # произвести какую-либо операцию (сложение)  
# с вторым и третьим элементами кортежа x1:  
c = x1[2] + x1[3]
```

```
[9]: 5
```

```
[10]: # обращение к элементам именованного кортежа x3:  
x3.a, x3.b, x3[2]
```

```
[10]: (2, 3, 3)
```

```
[6]: # проверка вхождения элементов tmp и 0 в кортеж x2  
# (два способа обращения к методу in()):  
in("tmp", x2), 0 in x2
```

```
[6]: (true, false)
```

Рис. 2: Примеры операций над кортежами

Словари

```
[33]: # создать словарь с именем phonebook:
phonebook = Dict("Иванов И.И." => ("867-5309", "333-5544"), "Бухгалтерия" => "555-2368")

[33]: Dict{String, Any} with 2 entries:
      "Бухгалтерия" => "555-2368"
      "Иванов И.И." => ("867-5309", "333-5544")

[34]: # вывести ключи словаря:
keys(phonebook)

[34]: KeySet for a Dict{String, Any} with 2 entries. Keys:
      "Бухгалтерия"
      "Иванов И.И."

[35]: # вывести значения элементов словаря:
values(phonebook)

[35]: ValueIterator for a Dict{String, Any} with 2 entries. Values:
      "555-2368"
      ("867-5309", "333-5544")

[36]: # вывести заданные в словаре пары "ключ - значение":
pairs(phonebook)

[36]: Dict{String, Any} with 2 entries:
      "Бухгалтерия" => "555-2368"
      "Иванов И.И." => ("867-5309", "333-5544")

[37]: # проверка вхождения ключа в словарь:
haskey(phonebook, "Иванов И.И.")

[37]: true

[38]: # добавить элемент в словарь:
phonebook["Сидоров П.С."] = "555-3344"

[38]: "555-3344"

[39]: # удалить ключ и связанные с ним значения из словаря
pop!(phonebook, "Иванов И.И.")

[39]: ("867-5309", "333-5544")

[40]: # Объединение словарей (функция merge()):
a = Dict{"foo" => 0.0, "bar" => 42.0};
b = Dict{"baz" => 17, "bar" => 13.0};
merge(a, b), merge(b, a)

[40]: (Dict{String, Real}{ "bar" => 13.0, "baz" => 17, "foo" => 0.0}, Dict{String, Real}{ "bar" => 42.0, "baz" => 17, "foo" => 0.0})
```

Рис. 3: Примеры словарей и операций над ними

Множества

```
[41]: # создать множество из четырёх целочисленных значений:  
A = Set([1, 3, 4, 5])
```

```
[41]: Set{Int64} with 4 elements:  
5  
4  
3  
1
```

```
[42]: # создать множество из 11 символьных значений:  
B = Set("abracadabra")
```

```
[42]: Set{Char} with 5 elements:  
'a'  
'd'  
'r'  
'k'  
'b'
```

```
[43]: # проверка эквивалентности двух множеств:  
S1 = Set([1,2]);  
S2 = Set([3,4]);  
issetequal(S1,S2)
```

```
[43]: false
```

```
[44]: S3 = Set([1,2,2,3,1,2,3,2,1]);  
S4 = Set([2,3,1]);  
issetequal(S3,S4)
```

```
[44]: true
```

Рис. 4: Примеры множеств и операций над ними

Множества

```
[45]: # объединение множеств:  
C=union(S1,S2)  
  
[45]: Set{Int64} with 4 elements:  
4  
2  
3  
1  
  
[46]: # пересечение множеств:  
D = intersect(S1,S3)  
  
[46]: Set{Int64} with 2 elements:  
2  
1  
  
[47]: # разность множеств:  
E = setdiff(S3,S1)  
  
[47]: Set{Int64} with 1 element:  
3  
  
[48]: # проверка вхождения элементов одного множества в другое:  
issubset(S1,S4)  
  
[48]: true  
  
[49]: # добавление элемента в множество:  
push!(S4, 99)  
  
[49]: Set{Int64} with 4 elements:  
2  
99  
3  
1  
  
[50]: # удаление последнего элемента множества:  
pop!(S4)  
  
[50]: 2
```

Рис. 5: Примеры множеств и операций над ними

Массивы

```
[51]: # создание пустого массива с абстрактным типом:  
empty_array_1 = []
```

```
[51]: Any[]
```

```
[52]: # создание пустого массива с конкретным типом:  
empty_array_2 = (Int64[])  
empty_array_3 = (Float64[])
```

```
[52]: Float64[]
```

```
[53]: # вектор-столбец:  
a = [1, 2, 3]
```

```
[53]: 3-element Vector{Int64}:  
      1  
      2  
      3
```

```
[54]: # вектор-строка:  
b = [1 2 3]
```

```
[54]: 1×3 Matrix{Int64}:  
      1  2  3
```

```
[55]: # многомерные массивы (матрицы):  
A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]  
B = [[1 2 3]; [4 5 6]; [7 8 9]]
```

```
[55]: 3×3 Matrix{Int64}:  
      1  2  3  
      4  5  6  
      7  8  9
```

Рис. 6: Примеры массивов

Массивы

```
[57]: # одномерный массив из 8 элементов (массив $1 \times 8$)
      # со значениями, случайно распределёнными на интервале [0, 1):
      c = rand(1,8)

[57]: 1×8 Matrix{Float64}:
      0.77315  0.704689  0.327715  0.137023  ...  0.0366656  0.425855  0.360563

[61]: # многомерный массив $2 \times 3$ (2 строки, 3 столбца) элементов
      # со значениями, случайно распределёнными на интервале [0, 1):
      C = rand(2,3)

[61]: 2×3 Matrix{Float64}:
      0.0947976  0.807583  0.439739
      0.499826  0.213197  0.767716

[59]: # трёхмерный массив
      D = rand(4, 3, 2)

[59]: 4×3×2 Array{Float64, 3}:
     [:, :, 1] =
      0.4238      0.512702  0.105071
      0.632824  0.0595373  0.299242
      0.397225  0.796443  0.962184
      0.604381  0.740173  0.599432

     [:, :, 2] =
      0.0451308  0.811325  0.795289
      0.455207  0.0222529  0.526592
      0.891493  0.258458  0.800133
      0.637477  0.253284  0.462157
```

Рис. 7: Примеры массивов

Массивы

```
[62]: # массив из квадратных корней всех целых чисел от 1 до 10:  
roots = [sqrt(i) for i in 1:10]
```

```
[62]: 10-element Vector{Float64}:  
 1.0  
 1.4142135623730951  
 1.7320508075688772  
 2.0  
 2.23606797749979  
 2.449489742783178  
 2.6457513110645907  
 2.8284271247461903  
 3.0  
 3.1622776601683795
```

```
[63]: # массив с элементами вида 3*x^2,  
# где x - нечётное число от 1 до 9 (включительно)  
ar_1 = [3*i^2 for i in 1:2:9]
```

```
[63]: 5-element Vector{Int64}:  
 3  
 27  
 75  
 147  
 243
```

```
[64]: # массив квадратов элементов, если квадрат не делится на 5 или 4:  
ar_2=[i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]
```

```
[64]: 4-element Vector{Int64}:  
 1  
 9  
 49  
 81
```

Рис. 8: Примеры массивов, заданных некоторыми функциями через включение

Массивы

```
[65]: # одномерный массив из пяти единиц:  
ones(5)
```

```
[65]: 5-element Vector{Float64}:  
 1.0  
 1.0  
 1.0  
 1.0  
 1.0
```

```
[66]: # двумерный массив 2x3 из единиц:  
ones(2,3)
```

```
[66]: 2x3 Matrix{Float64}:  
 1.0  1.0  1.0  
 1.0  1.0  1.0
```

```
[67]: # одномерный массив из 4 нулей:  
zeros(4)
```

```
[67]: 4-element Vector{Float64}:  
 0.0  
 0.0  
 0.0  
 0.0
```

```
[68]: # заполнить массив 3x2 цифрами 3.5  
fill(3.5,(3,2))
```

```
[68]: 3x2 Matrix{Float64}:  
 3.5  3.5  
 3.5  3.5  
 3.5  3.5
```

Рис. 9: Некоторые операции для работы с массивами

Массивы

```
[69]: # заполнение массива посредством функции repeat():
      repeat([1,2],3,3)
      repeat([1 2],3,3)

[69]: 3×6 Matrix{Int64}:
      1  2  1  2  1  2
      1  2  1  2  1  2
      1  2  1  2  1  2

[70]: # преобразование одномерного массива из целых чисел от 1 до 12
      # в двумерный массив 2х6
      a = collect(1:12)
      b = reshape(a,(2,6))

[70]: 2×6 Matrix{Int64}:
      1  3  5  7  9  11
      2  4  6  8  10  12

[71]: # транспонирование
      b'

[71]: 6×2 adjoint{::Matrix{Int64}} with eltype Int64:
      1  2
      3  4
      5  6
      7  8
      9  10
     11  12
```

Рис. 10: Некоторые операции для работы с массивами

Массивы

```
[72]: # транспонирование
      c = transpose(b)

[72]: 6×2 transpose(::Matrix{Int64}) with eltype Int64:
      1  2
      3  4
      5  6
      7  8
      9 10
     11 12

[73]: # массив 10×5 целых чисел в диапазоне [10, 20]:
      ar = rand(10:20, 10, 5)

[73]: 10×5 Matrix{Int64}:
      19 16 16 17 15
      10 18 12 12 20
      17 12 10 16 10
      19 18 10 10 14
      11 12 20 13 14
      12 19 16 10 19
      20 13 20 17 12
      12 12 15 18 15
      20 19 16 14 12
      15 18 11 16 10

[74]: # выбор всех значений строки в столбце 2:
      ar[:, 2]

[74]: 10-element Vector{Int64}:
      16
      18
      12
      18
      12
      19
      13
      12
      19
      18
```

Рис. 11: Некоторые операции для работы с массивами

Массивы

```
[75]: # выбор всех значений в столбцах 2 и 5:  
ar[:, [2, 5]]
```

```
[75]: 10x2 Matrix{Int64}:  
 16 15  
 18 20  
 12 10  
 18 14  
 12 14  
 19 19  
 13 12  
 12 15  
 19 12  
 18 10
```

```
[76]: # все значения строк в столбцах 2, 3 и 4:  
ar[:, 2:4]
```

```
[76]: 10x3 Matrix{Int64}:  
 16 16 17  
 18 12 12  
 12 10 16  
 18 10 10  
 12 20 13  
 19 16 10  
 13 20 17  
 12 15 18  
 19 16 14  
 18 11 16
```

```
[77]: # значения в строках 2, 4, 6 и 8 столбцах 1 и 5:  
ar[[2, 4, 6], [1, 5]]
```

```
[77]: 3x2 Matrix{Int64}:  
 10 20  
 19 14  
 12 19
```

Рис. 12: Некоторые операции для работы с массивами

Массивы

```
[78]: # значения в строке 1 от столбца 3 до последнего столбца:  
ar[1, 3:end]
```

```
[78]: 3-element Vector{Int64}:  
      16  
      17  
      15
```

```
[79]: # сортировка по столбцам:  
sort(ar,dims=1)
```

```
[79]: 10x5 Matrix{Int64}:  
      10  12  10  10  10  
      11  12  10  10  10  
      12  12  11  12  12  
      12  13  12  13  12  
      15  16  15  14  14  
      17  18  16  16  14  
      19  18  16  16  15  
      19  18  16  17  15  
      20  19  20  17  19  
      20  19  20  18  20
```

```
[80]: # сортировка по строкам:  
sort(ar,dims=2)
```

```
[80]: 10x5 Matrix{Int64}:  
      15  16  16  17  19  
      10  12  12  18  20  
      10  10  12  16  17  
      10  10  14  18  19  
      11  12  13  14  20  
      10  12  16  19  19  
      12  13  17  20  20  
      12  12  15  15  18  
      12  14  16  19  20  
      10  11  15  16  18
```

Рис. 13: Некоторые операции для работы с массивами

Массивы

```
[81]: # поэлементное сравнение с числом
      # (результат - массив логических значений):
      ar .> 14

[81]: 10x5 BitMatrix:
      1 1 1 1 1
      0 1 0 0 1
      1 0 0 1 0
      1 1 0 0 0
      0 0 1 0 0
      0 1 1 0 1
      1 0 1 1 0
      0 0 1 1 1
      1 1 1 0 0
      1 1 0 1 0

[82]: # возврат индексов элементов массива, удовлетворяющих условию:
      findall(ar .> 14)

[82]: 27-element Vector{CartesianIndex{2}}:
      CartesianIndex(1, 1)
      CartesianIndex(3, 1)
      CartesianIndex(4, 1)
      CartesianIndex(7, 1)
      CartesianIndex(9, 1)
      CartesianIndex(10, 1)
      CartesianIndex(1, 2)
      CartesianIndex(2, 2)
      CartesianIndex(4, 2)
      CartesianIndex(6, 2)
      CartesianIndex(9, 2)
      CartesianIndex(10, 2)
      CartesianIndex(1, 3)
      ⋮
      CartesianIndex(7, 3)
      CartesianIndex(8, 3)
      CartesianIndex(9, 3)
      CartesianIndex(1, 4)
      CartesianIndex(3, 4)
      CartesianIndex(7, 4)
      CartesianIndex(8, 4)
      CartesianIndex(10, 4)
      CartesianIndex(1, 5)
      CartesianIndex(2, 5)
      CartesianIndex(6, 5)
      CartesianIndex(8, 5)
```

Рис. 14: Некоторые операции для работы с массивами

Самостоятельная работа

```
•[83]: println("=== Задание 1: Множества ===")

A = Set([0, 3, 4, 9])
B = Set([1, 3, 4, 7])
C = Set([0, 1, 2, 4, 7, 8, 9])

P = union(intersect(A, B), intersect(A, B), intersect(A, C), intersect(B, C))
println(P)

Set([0, 4, 7, 9, 3, 1])

•[87]: println("\n=== Задание 2 (1): Множества строк ===")

set1 = Set(["dev", "sport", "study"])
set2 = Set(["sport", "study", "games"])
intersection = intersect(set1, set2)
println(intersection)

=== Задание 2: Множества разных типов ===
Set(["sport", "study"])

[89]: println("\n=== Задание 2 (2): Множества чисел ===")

set3 = Set([10, 20, 30])
set4 = Set([30, 40, 50])
difference = setdiff(set3, set4)
println(difference)

=== Задание 2 (2): Множества чисел ===
Set([20, 10])
```

Рис. 15: Решение заданий №1 и №2

Самостоятельная работа

```
[90]: println("\n=== Задание 3: Массивы ===")
      N = 25
      arr1 = collect(1:N)
      println(arr1)

      === Задание 3: Массивы ===
      [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 2
      2, 23, 24, 25]

[91]: arr2 = collect(N:-1:1)
      println(arr2)

      [25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6,
      5, 4, 3, 2, 1]

[92]: arr3 = vcat(collect(1:N), collect((N-1):-1:1))
      println(arr3)

      [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 2
      2, 23, 24, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9,
      8, 7, 6, 5, 4, 3, 2, 1]

[93]: tmp = [5, 10, 15]
      println(tmp)

      [5, 10, 15]
```

Рис. 16: Выполнение подпунктов задания №3

Самостоятельная работа

```
[94]: arr4 = fill(tmp[1], 10)
      println(arr4)
      [5, 5, 5, 5, 5, 5, 5, 5, 5, 5]
```

```
[95]: arr5 = repeat(tmp, 10)
      println(arr5)
      [5, 10, 15, 5, 10, 15, 5, 10, 15, 5, 10, 15, 5, 10, 15, 5, 10, 15, 5, 10, 15,
      5, 10, 15, 5, 10, 15, 5, 10, 15]
```

```
[96]: arr6 = vcat(fill(tmp[1], 11), fill(tmp[2], 10), fill(tmp[3], 10))
      println(arr6)
      [5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 15,
      15, 15, 15, 15, 15, 15, 15]
```

```
[97]: arr7 = vcat(fill(tmp[1], 10), fill(tmp[2], 20), fill(tmp[3], 30))
      println(arr7)
      [5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10,
      10, 10, 10, 10, 10, 10, 10, 10, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 1
      5, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15]
```

Рис. 17: Выполнение подпунктов задания №3

Самостоятельная работа

```
[114]: tmp = [5, 10, 15]

resultArr = [2^tmp[1] for i in 1:3]
resultArr = vcat(resultArr, repeat([2^tmp[3]], 4))

resultStr = join(resultArr, "")
count_6 = count(x -> x == '6', resultStr)

println("Массив степеней: ", resultArr)
println("Количество цифр 6: ", count_6)

Массив степеней: [32, 32, 32, 32768, 32768, 32768, 32768]
Количество цифр 6: 4
```

```
•[118]: using Statistics

x_range = 3:0.1:6
y = [exp(x) * cos(x) for x in x_range]
println(mean(y))

3.11 Вектор степеней: длина = 24
x_vals (0.1^i): 12 элементов
y_vals (0.2^j): 12 элементов
```

Рис. 18: Выполнение подпунктов задания №3 21/26

Самостоятельная работа

```
[125]: x_vals = collect(3:3:36) * 0.1
       y_vals = collect(1:3:34) * 0.2
       vector = [(x,y) for x in x_vals, y in y_vals]
       println(vector)

[(0.30000000000000004, 0.2) (0.30000000000000004, 0.8) (0.30000000000000004, 1.4000000000000001) (0.30000000000000004, 2.0) (0.30000000000000004, 2.6) (0.30000000000000004, 3.2) (0.30000000000000004, 3.8000000000000003) (0.30000000000000004, 4.4) (0.30000000000000004, 5.0) (0.30000000000000004, 5.6000000000000005) (0.30000000000000004, 6.2) (0.30000000000000004, 6.800000000000001) (0.6000000000000001, 0.2) (0.6000000000000001, 0.8) (0.6000000000000001, 1.4000000000000001) (0.6000000000000001, 2.0) (0.6000000000000001, 2.6) (0.6000000000000001, 3.2) (0.6000000000000001, 3.8000000000000003) (0.6000000000000001, 4.4) (0.6000000000000001, 5.0) (0.6000000000000001, 5.6000000000000005) (0.6000000000000001, 6.2) (0.6000000000000001, 6.800000000000001) (0.9, 0.2) (0.9, 0.8) (0.9, 1.4000000000000001) (0.9, 2.0) (0.9, 2.6) (0.9, 3.2) (0.9, 3.8000000000000003) (0.9, 4.4) (0.9, 5.0) (0.9, 5.6000000000000005) (0.9, 6.2) (0.9, 6.800000000000001) (1.2000000000000002, 0.2) (1.2000000000000002, 0.8) (1.2000000000000002, 1.4000000000000001) (1.2000000000000002, 2.0) (1.2000000000000002, 2.6) (1.2000000000000002, 3.2) (1.2000000000000002, 3.8000000000000003) (1.2000000000000002, 4.4) (1.2000000000000002, 5.0) (1.2000000000000002, 5.6000000000000005) (1.2000000000000002, 6.2) (1.2000000000000002, 6.800000000000001) (1.5, 0.2) (1.5, 0.8) (1.5, 1.4000000000000001) (1.5, 2.0) (1.5, 2.6) (1.5, 3.2) (1.5, 3.8000000000000003) (1.5, 4.4) (1.5, 5.0) (1.5, 5.6000000000000005) (1.5, 6.2) (1.5, 6.800000000000001) (1.8, 0.2) (1.8, 0.8) (1.8, 1.4000000000000001) (1.8, 2.0) (1.8, 2.6) (1.8, 3.2) (1.8, 3.8000000000000003) (1.8, 4.4) (1.8, 5.0) (1.8, 5.6000000000000005) (1.8, 6.2) (1.8, 6.800000000000001) (2.1, 0.2) (2.1, 0.8) (2.1, 1.4000000000000001) (2.1, 2.0) (2.1, 2.6) (2.1, 3.2) (2.1, 3.8000000000000003) (2.1, 4.4) (2.1, 5.0) (2.1, 5.6000000000000005) (2.1, 6.2) (2.1, 6.800000000000001) (2.4000000000000004, 0.2) (2.4000000000000004, 0.8) (2.4000000000000004, 1.4000000000000001) (2.4000000000000004, 2.0) (2.4000000000000004, 2.6) (2.4000000000000004, 3.2) (2.4000000000000004, 3.8000000000000003) (2.4000000000000004, 4.4) (2.4000000000000004, 5.0) (2.4000000000000004, 5.6000000000000005) (2.4000000000000004, 6.2) (2.4000000000000004, 6.800000000000001) (2.7, 0.2) (2.7, 0.8) (2.7, 1.4000000000000001) (2.7, 2.0) (2.7, 2.6) (2.7, 3.2) (2.7, 3.8000000000000003) (2.7, 4.4) (2.7, 5.0) (2.7, 5.6000000000000005) (2.7, 6.2) (2.7, 6.800000000000001) (3.0, 0.2) (3.0, 0.8) (3.0, 1.4000000000000001) (3.0, 2.0) (3.0, 2.6) (3.0, 3.2) (3.0, 3.8000000000000003) (3.0, 4.4) (3.0, 5.0) (3.0, 5.6000000000000005) (3.0, 6.2) (3.0, 6.800000000000001) (3.3000000000000003, 0.2) (3.3000000000000003, 0.8) (3.3000000000000003, 1.4000000000000001) (3.3000000000000003, 2.0) (3.3000000000000003, 2.6) (3.3000000000000003, 3.2) (3.3000000000000003, 3.8000000000000003) (3.3000000000000003, 4.4) (3.3000000000000003, 5.0) (3.3000000000000003, 5.6000000000000005) (3.3000000000000003, 6.2) (3.3000000000000003, 6.800000000000001) (3.6, 0.2) (3.6, 0.8) (3.6, 1.4000000000000001) (3.6, 2.0) (3.6, 2.6) (3.6, 3.2) (3.6, 3.8000000000000003) (3.6, 4.4) (3.6, 5.0) (3.6, 5.6000000000000005) (3.6, 6.2) (3.6, 6.800000000000001)]

[126]: M = 25
       vector12 = [2^i / i for i in 1:M]
       println(vector12)

[2.0, 2.0, 2.6666666666666665, 4.0, 6.4, 10.666666666666666, 18.285714285714285, 32.0, 56.888888888888886, 102.4, 186.1818181818182, 341.3333333333333, 630.1538461538462, 1170.2857142857142, 2184.5333333333333, 4096.0, 7710.117647058823, 14563.555555555555, 27594.105263157893, 52428.8, 99864.38095238095, 190650.18181818182, 364722.0869565217, 699050.6666666666, 1.34217728e6]
```

Рис. 19: Выполнение подпунктов задания №3

Самостоятельная работа

```
[127]: N_names = 30
arr13 = ["fn%i" for i in 1:N_names]
println(arr13)

["fn1", "fn2", "fn3", "fn4", "fn5", "fn6", "fn7", "fn8", "fn9", "fn10", "fn11", "fn12", "fn13", "fn14", "fn15", "fn16", "fn17", "fn18", "fn19", "fn20", "fn21", "fn22", "fn23", "fn24", "fn25", "fn26", "fn27", "fn28", "fn29", "fn30"]

[142]: using Random
x = rand(0:999, 250)
y = rand(0:999, 250)

# Выбор y > 600
filtered_y = y[y .> 600]
println("Элементы y > 600: ", filtered_y)

# Соответствующие элементы x
corresponding_x = x[findall(y .> 600)]
println("\n")
println("Соответствующие элементы x: ", corresponding_x)

# Различные операции
sum_res = sum(exp.(-x[2:end] + x[1:end-1]).+ 10)
println("\n")
println("Сумма: ", sum_res)

# Уникальные элементы x
uniqueX = unique(x)
println("\n")
println("Уникальные элементы x: ", uniqueX)

Элементы y > 600: [833, 632, 871, 997, 759, 866, 630, 999, 767, 705, 807, 901, 610, 844, 978, 795, 888, 842, 920, 919, 841, 840, 709, 812, 779, 718, 951, 989, 633, 742, 771, 846, 955, 820, 876, 746, 691, 986, 889, 997, 893, 632, 739, 705, 862, 628, 782, 668, 820, 623, 923, 933, 837, 914, 725, 766, 873, 909, 999, 929, 978, 894, 971, 855, 702, 963, 764, 609, 688, 833, 862, 988, 935, 700, 806, 608, 934, 814, 680, 721, 707, 686, 640, 859, 706, 876, 875, 903, 690, 711]

Соответствующие элементы x: [943, 660, 639, 392, 24, 333, 562, 721, 169, 684, 697, 269, 105, 457, 536, 311, 768, 645, 209, 210, 33, 704, 242, 139, 377, 436, 30, 489, 261, 546, 676, 809, 94, 392, 269, 947, 25, 81, 813, 486, 385, 187, 755, 641, 965, 703, 367, 209, 102, 355, 989, 508, 879, 400, 506, 301, 198, 549, 271, 111, 267, 449, 510, 779, 163, 540, 203, 720, 405, 210, 993, 684, 893, 675, 462, 663, 382, 217, 44, 402, 18, 752, 296, 323, 924, 836, 521, 450, 100, 766]

Сумма: Inf

Уникальные элементы x: [581, 943, 690, 137, 660, 905, 866, 639, 772, 392, 320, 521, 24, 804, 220, 102, 333, 83, 254, 562, 739, 721, 169, 684, 697, 986, 117, 269, 105, 457, 66, 437, 85, 536, 244, 311, 398, 617, 594, 768, 412, 22, 403, 645, 84, 826, 209, 56, 653, 210, 86, 33, 773, 262, 356, 704, 848, 242, 139, 16, 526, 45, 760, 950, 377, 827, 901, 216, 148, 436, 580, 30, 463, 489, 358, 40, 261, 546, 676, 809, 197, 294, 48, 94, 978, 947, 25, 0, 430, 94, 5, 875, 81, 940, 400, 813, 486, 385, 328, 187, 755, 134, 641, 965, 703, 367, 414, 292, 3, 734, 260, 554, 355, 989, 470, 567, 52, 515, 508, 146, 879, 419, 506, 301, 339, 198, 904, 444, 849, 332, 832, 906, 582, 164, 549, 380, 975, 141, 271, 76, 520, 111, 74, 14, 267, 62, 449, 388, 510, 133, 744, 779, 434, 171, 31, 391, 163, 540, 619, 203, 720, 319, 599, 952, 405, 680, 253, 671, 543, 993, 103, 982, 925, 893, 675, 462, 12, 663, 129, 325, 408, 382, 217, 166, 491, 593, 330, 44, 402, 235, 545, 18, 309, 570, 752, 830, 615, 485, 179, 296, 612, 323, 447, 924, 836, 75, 109, 942, 194, 458, 100, 531, 743, 156, 410, 914, 766]
```

Рис. 20: Выполнение подпунктов задания №3

AAA

```
[143]: A = Set([0, 3, 4, 9])
      B = Set([1, 3, 4, 7])
      C = Set([0, 1, 2, 4, 7, 8, 9])

      P = union(intersect(A, B), intersect(A, C), intersect(B, C))
      println(P)

      Set([0, 4, 7, 9, 3, 1])

[145]: set1 = Set(["dev", "sport", "study"])
      set2 = Set(["sport", "study", "games"])
      intersection = intersect(set1, set2)
      println(intersection)

      Set(["sport", "study"])

[146]: set3 = Set([10,20,30])
      set4 = Set([30,40,50])
      difference = setdiff(set3, set4)
      println(difference)

      Set([20, 10])
```

Рис. 21: Решение заданий №4, №5 и №6

Вывод

- В ходе выполнения лабораторной работы были изучены несколько структур данных, реализованных в Julia, а также научились применять их и операции над ними для решения задач.

Список литературы. Библиография

[1] Julia Documentation: <https://docs.julialang.org/en/v1/>